Experiment 14

Pro-Code Extension Using Zoho Catalyst

Aim

To study and implement pro-code extension using Zoho Catalyst to extend the Student Admission Enquiry Management (Advanced Edition) application with custom backend logic, APIs and database operations beyond the built-in capabilities of Zoho Creator.

Introduction

Zoho Catalyst is a pro-code development platform that supports custom backend functions, data storage, APIs, authentication and integrations. For the Student Admission application, a Catalyst function can calculate a detailed application-fee estimate or provide a custom backend service beyond the built-in Creator features.

Procedure

1. Open the Zoho Catalyst website and sign in with the Zoho account.

2. Create a new Catalyst project, for example StudentAdmission-Project.

3. Open CloudScale and review Authentication, Data Store, File Store, API Gateway and other components.

4. Install the Catalyst CLI and initialize the project.

5. Create a Data Store table such as AdmissionFeeEstimate.

6. Add fields for Admission_Enquiry_ID, Application_Fee, Scholarship_Percent and Net_Fee.

7. Create a serverless function and API endpoint for custom fee calculation.

8. Connect the Creator application to the deployed API using invokeUrl and a secure connection.

CLI Commands

```
npm install -g zcatalyst-cli
catalyst login
catalyst init
catalyst deploy
```

Data Store Table: AdmissionFeeEstimate

Column	Type	Purpose
ROWID / system columns	System	Created automatically
Admission_Enquiry_ID	Text	Admission enquiry reference
Application_Fee	Decimal	Original application fee
Scholarship_Percent	Decimal	Eligible scholarship percentage
Net_Fee	Decimal	Calculated fee returned to Creator

Sample Function: Admission Fee Calculator (Node.js)

```
const express = require("express");
const app = express();
app.use(express.json());

app.post("/fee", (req, res) => {
  const { fee, scholarshipPercent = 0 } = req.body;
  const discount = Number(fee) * Number(scholarshipPercent) / 100;
  const total = Number(fee) - discount;
  res.json({ fee, discount, total });
});

module.exports = app;
```

Process Diagram

Figure 14.1: Pro-code extension architecture for Student Admission

Zoho Creator Form	Catalyst API Gateway	Serverless Fee Function	Data Store

Function Test (Expected)

```
POST /fee
{ "fee": 5000, "scholarshipPercent": 15 }

Response
{ "fee": 5000, "discount": 750, "total": 4250 }
```

Deployment Checklist

• Install the Catalyst CLI and log in.

• Initialize the project and add the function.

• Deploy with catalyst deploy and note the API Gateway URL.

• Call the URL from Deluge using invokeUrl and a secure connection.

• Verify the returned fee before displaying it in the admission report.

Result

The Zoho Catalyst project and its Data Store were configured conceptually, and the pro-code extension approach for custom Student Admission fee processing was studied.

Conclusion

Pro-code extension combines low-code development with traditional programming, enabling custom backend services, external integrations and advanced functionality beyond standard Creator features.